01211373 Machine Learning and Programming for Industry

Lecture 11 Supplement 1

Transformers and attention for machine signals

Self-attention, the transformer block, pre-training and what changed since BERT and GPT — all on one CNC turning cell

01 Why attention: the recurrent bottleneck

02 Self-attention, Q/K/V and the $\sqrt{d_k}$ scaling

03 Multi-head attention, the Pre-LN block, causal masks

04 Positional information and patching a signal into tokens

05 Transformer vs. LSTM on the same parts

06 Encoder, decoder and today's LLMs

07 Pre-training on unlabeled cycles, adapting with few labels

08 Reading attention maps critically

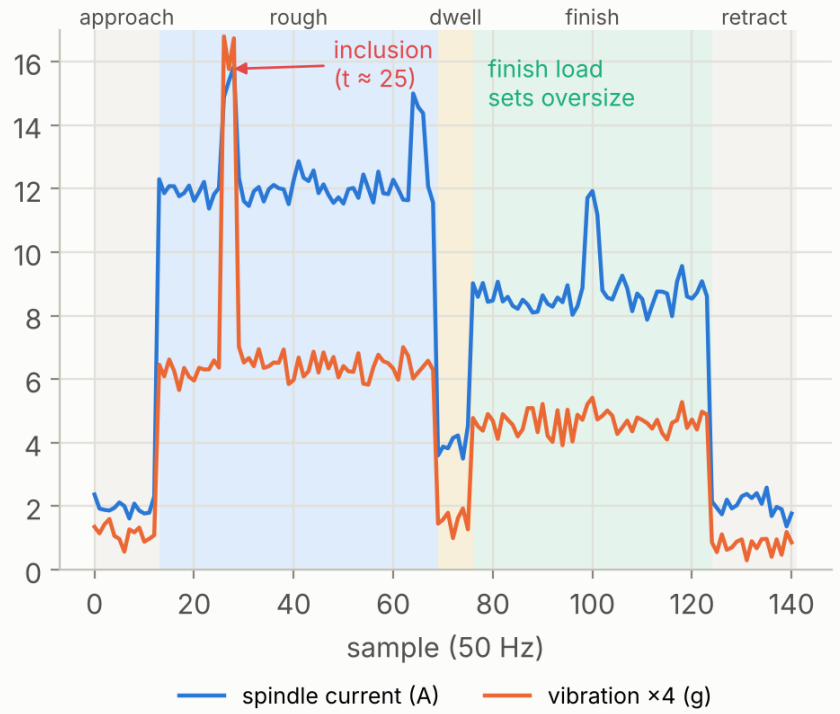
Companion notebook

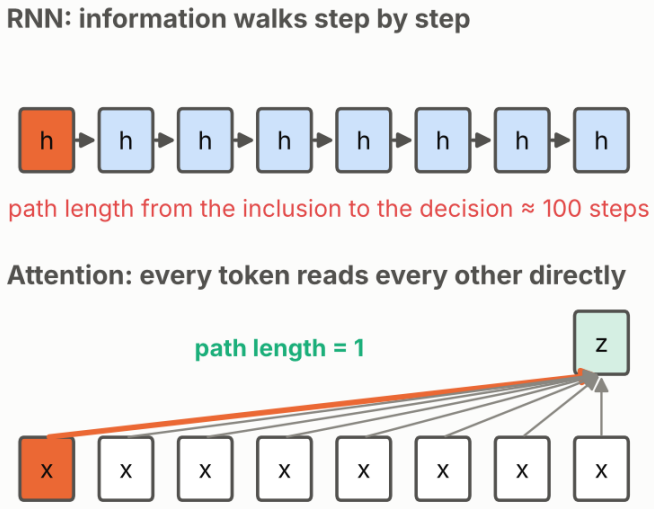
`Mlpi26_lect11_nbsup1.ipynb`

executed end to end, ~4 min on 2 CPU cores, no downloads

Attention gives every token a one-step path to every other

- An RNN must carry the inclusion through ~100 recurrent steps before the finish pass decides the label — the path length grows with distance.
- Bahdanau attention (2015) let a decoder read all encoder states directly; the transformer (2017) keeps only that and drops the recurrence.

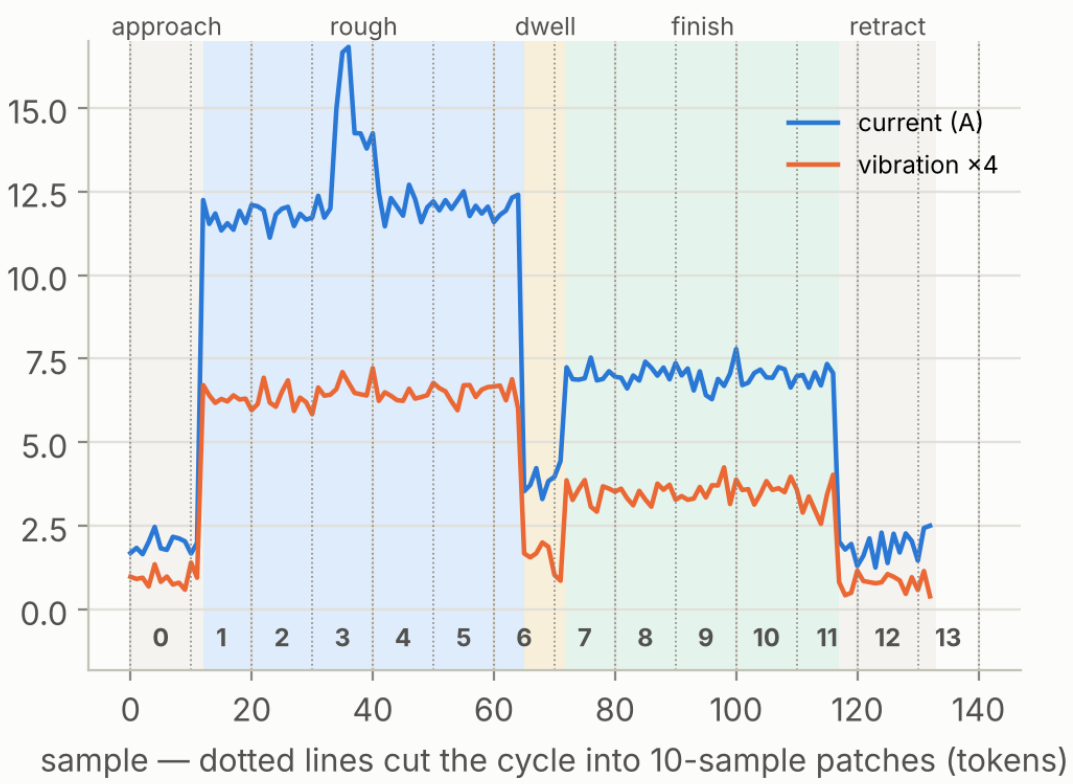


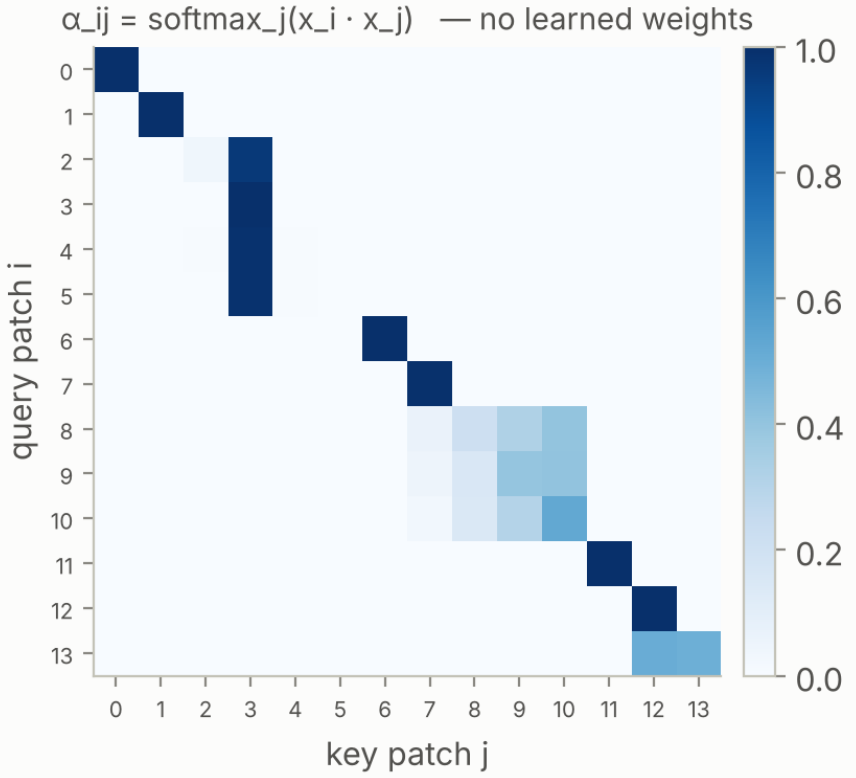


The cost of a long dependency moves from "how far back" to "how many tokens": path length 1, all positions computed in parallel.

Self-attention with no parameters is a similarity-weighted average

- Cut the cycle into 10-sample patches (tokens). Each output is $z_i = \Sigma_j \alpha_{ij} x_j$ with $\alpha_{ij} = \text{softmax}_j(x_i \cdot x_j)$: rows sum to 1, and the loop and matrix forms agree to 2.4e-7.

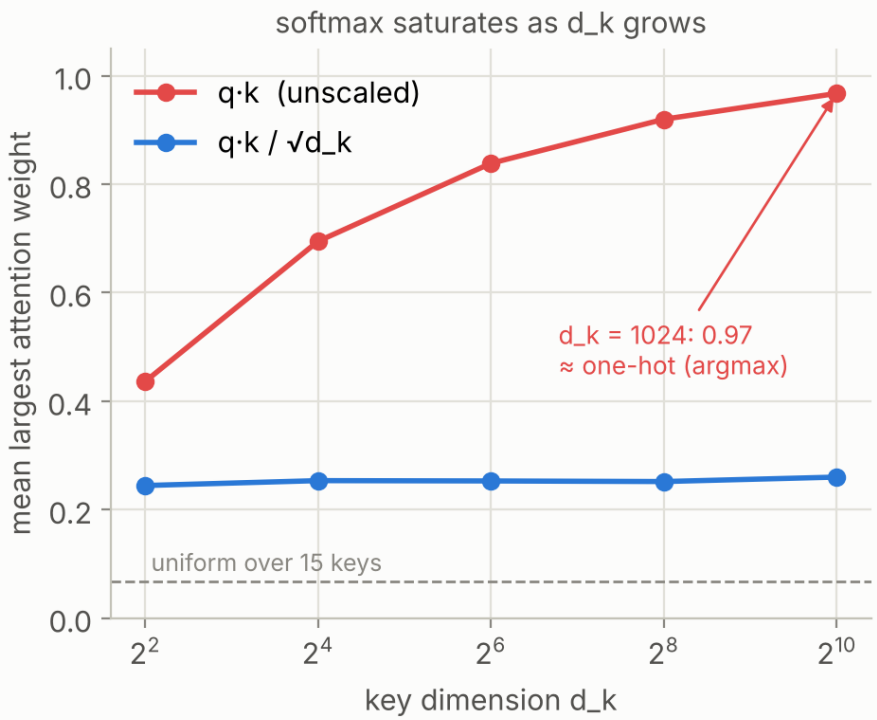


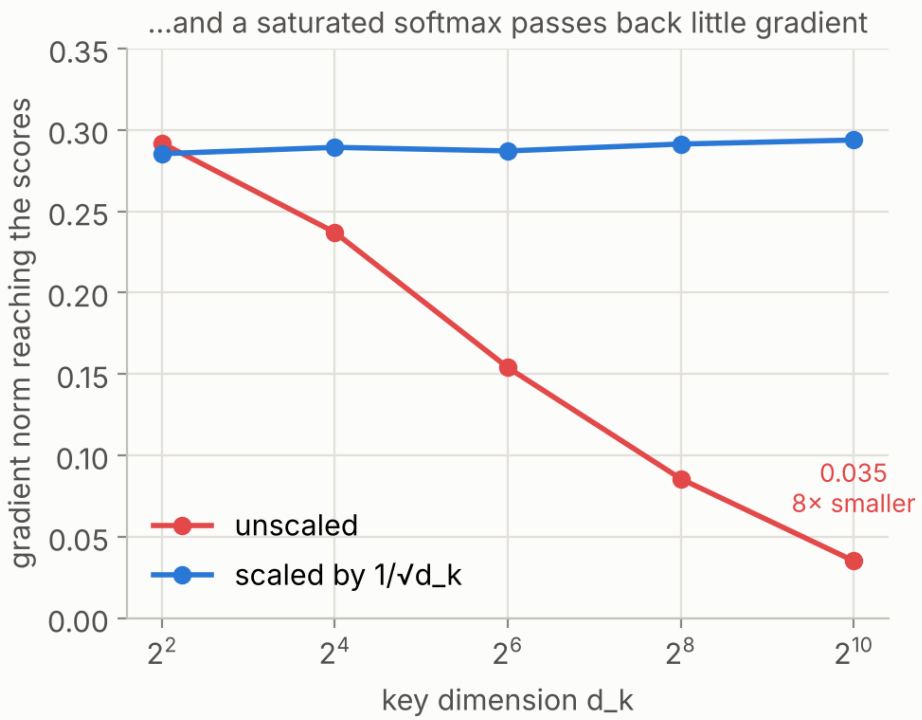


Phases emerge without training, but raw dot products favour large-norm tokens — patches 2–5 all stare at the spike in patch 3. Learned projections fix this.

Queries, keys, values — and why scores are divided by $\sqrt{d_k}$

- Q = XW_q (what I look for), K = XW_k (what I contain), V = XW_v (what I hand over): $Z = \text{softmax}(QK^T / \sqrt{d_k})\,V$.
- The score q·k sums d_k products, so its spread grows like $\sqrt{d_k}$; a measurement with random unit-variance vectors over 15 keys:

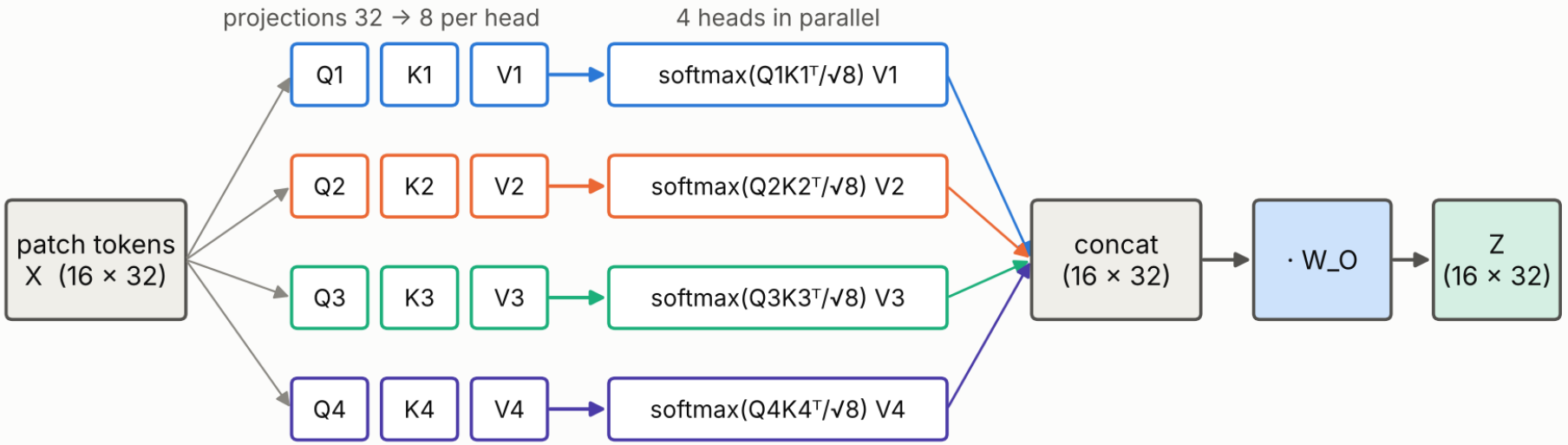




Unscaled at d_k = 1024: largest weight 0.97 and gradient 0.035 (vs 0.294 scaled). Scaling keeps attention soft and trainable.

Multi-head attention: four small attentions in parallel

- Each of h = 4 heads works in 8 of the 32 dimensions, so one head can track the inclusion while another tracks the finish load — at the cost of one wide head.

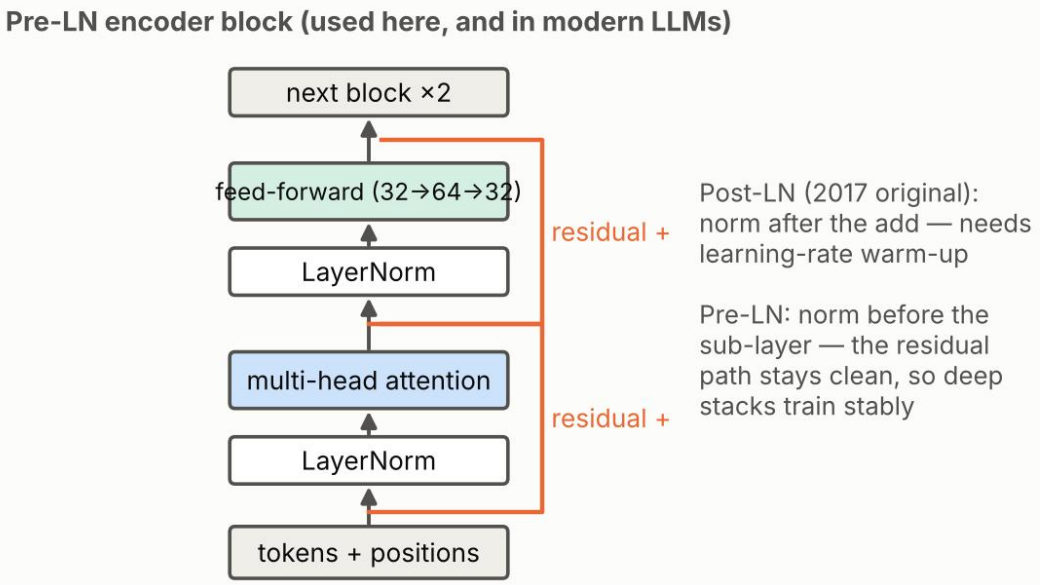


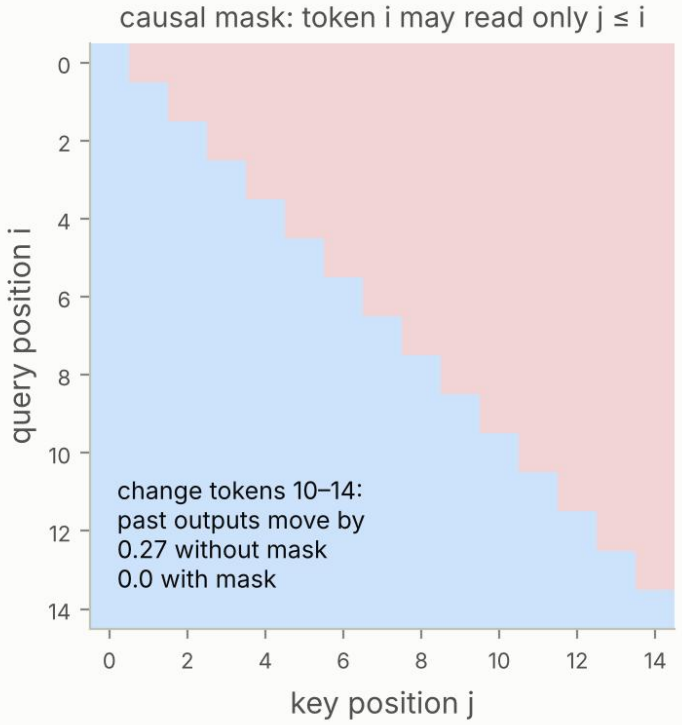

```
for i in range(h):                                      # by hand, using nn.MultiheadAttention's weights
    sl = slice(i*dh, (i+1)*dh)
    A  = torch.softmax(Q[..., sl] @ K[..., sl].transpose(1, 2) / dh**0.5, -1)
    heads.append(A @ V[..., sl])
out = torch.cat(heads, -1) @ W_O.T + b_O        # max |diff| vs PyTorch = 8.9e-8
```

nn.MultiheadAttention is these five lines: split into heads, attend, concatenate, mix with W_O.

The block: attention, feed-forward, residuals, LayerNorm — and masks

- LayerNorm normalises each token over its features (no dependence on batch or padding). Pre-LN puts it before each sub-layer, which modern models use because deep stacks train without warm-up.
- A decoder masks the future so next-step prediction cannot cheat by copying.





Test, don't trust: changing tokens 10–14 moves earlier outputs by 0.27 without the mask and by exactly 0.0 with it.

POSITIONAL INFORMATION

Self-attention cannot see order until you give it positions

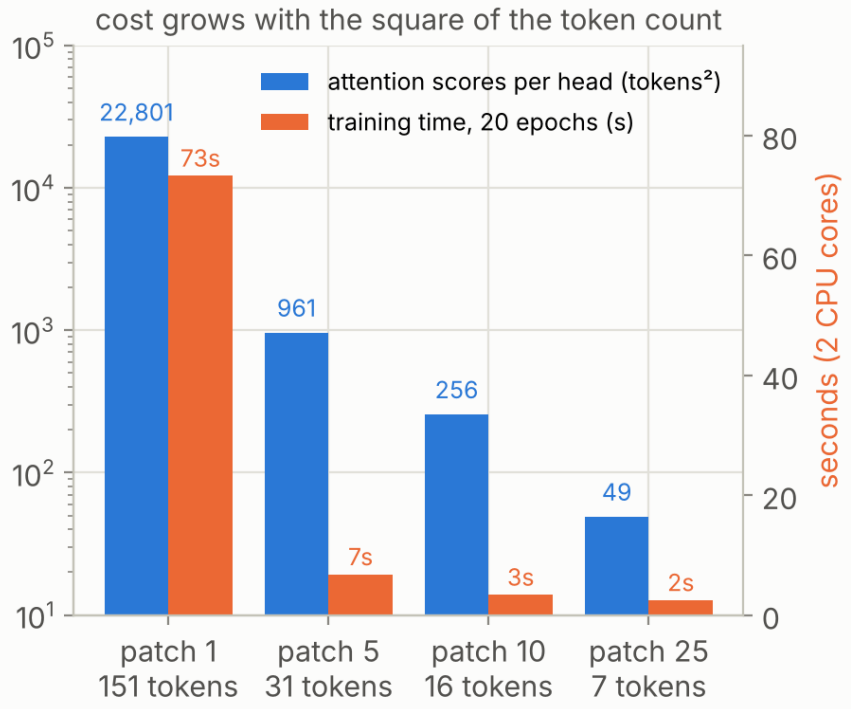
- Shuffle the patches of a cycle: without positional embeddings the logits change by 4.8e-7 (round-off) — the model sees a bag of patches.
- Options: sinusoidal (original), learned per position (BERT, here), rotary RoPE (most current LLMs).

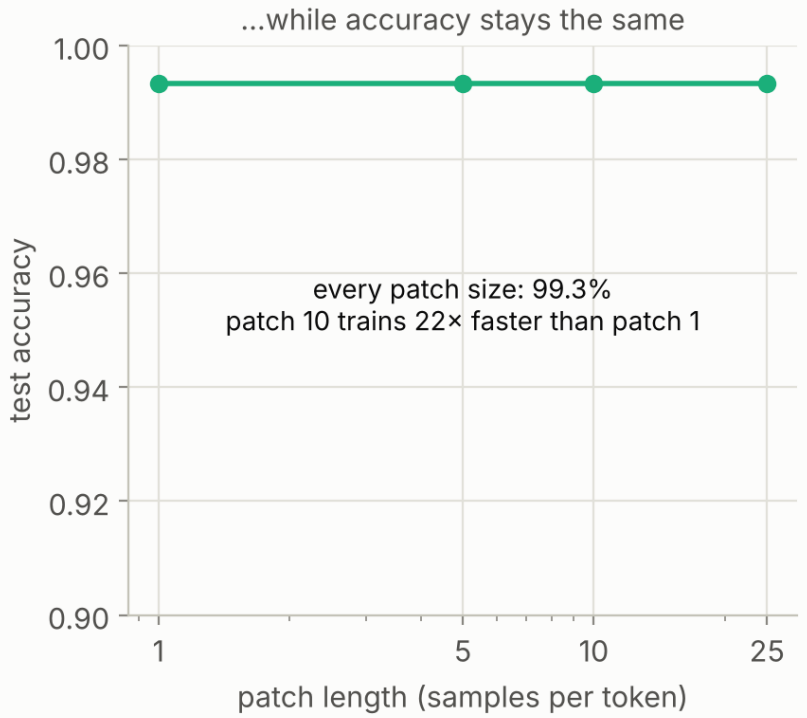


Diameter task: no-PE model still scores 99.3% — op codes reveal the phase. Scrambled-logger task: 51.5% vs 99.8%.

Turning a signal into tokens: patch it

- A token per 50 Hz sample gives 151 tokens and 22,801 scores per head per layer; attention cost grows with the square of the token count.
- Patching consecutive samples into one token (PatchTST, and ViT's image patches) shrinks the sequence and gives each token local context.

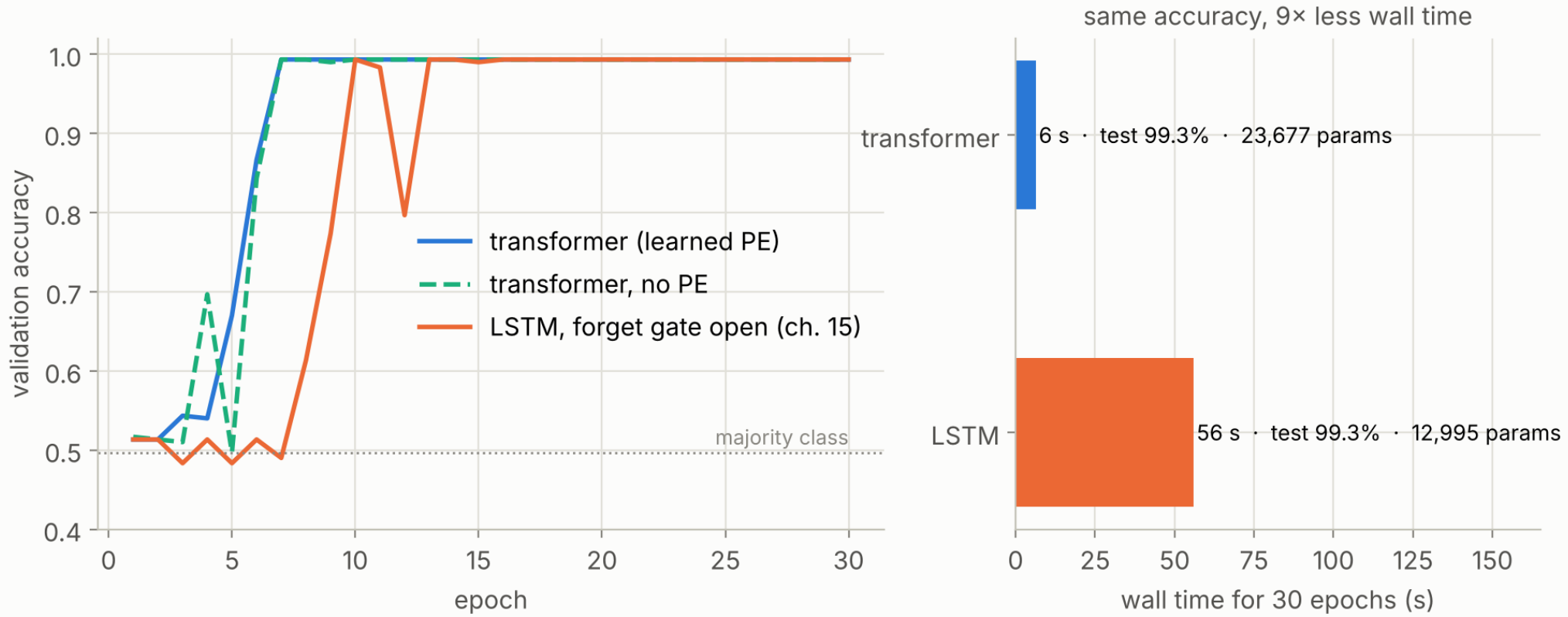




Same 99.3% at every patch size; patch 10 trains 22× faster than patch 1. Choose the patch to fit the shortest event you need.

Same parts, same accuracy — a fraction of the training time

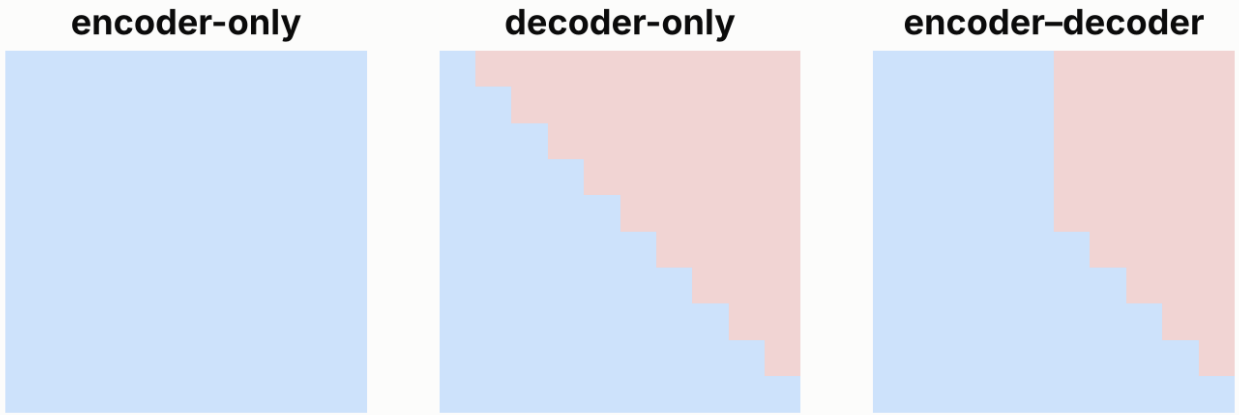
- Two Pre-LN blocks, 4 heads, d = 32, [CLS] head (23,677 parameters) vs. last week's open-gate LSTM (12,995): 30 epochs each, 800 / 300 / 600 split.



Both reach 99.3% test accuracy; the transformer is past 95% validation by epoch 7 (LSTM: 10) and trains 9× faster because all tokens are processed at once.

Three families, one block — and what changed since the book

rows = queries, columns = keys; blue = may attend



encoder-only	decoder-only	encoder–decoder
BERT RoBERTa, DistilBERT	GPT family Llama, Claude, …	T5, BART, Whisper
masked-token prediction reads both directions → classify, tag, embed	next-token prediction reads only the past → generate, chat, few-shot	encoder reads the input, decoder generates output → translate, summarise

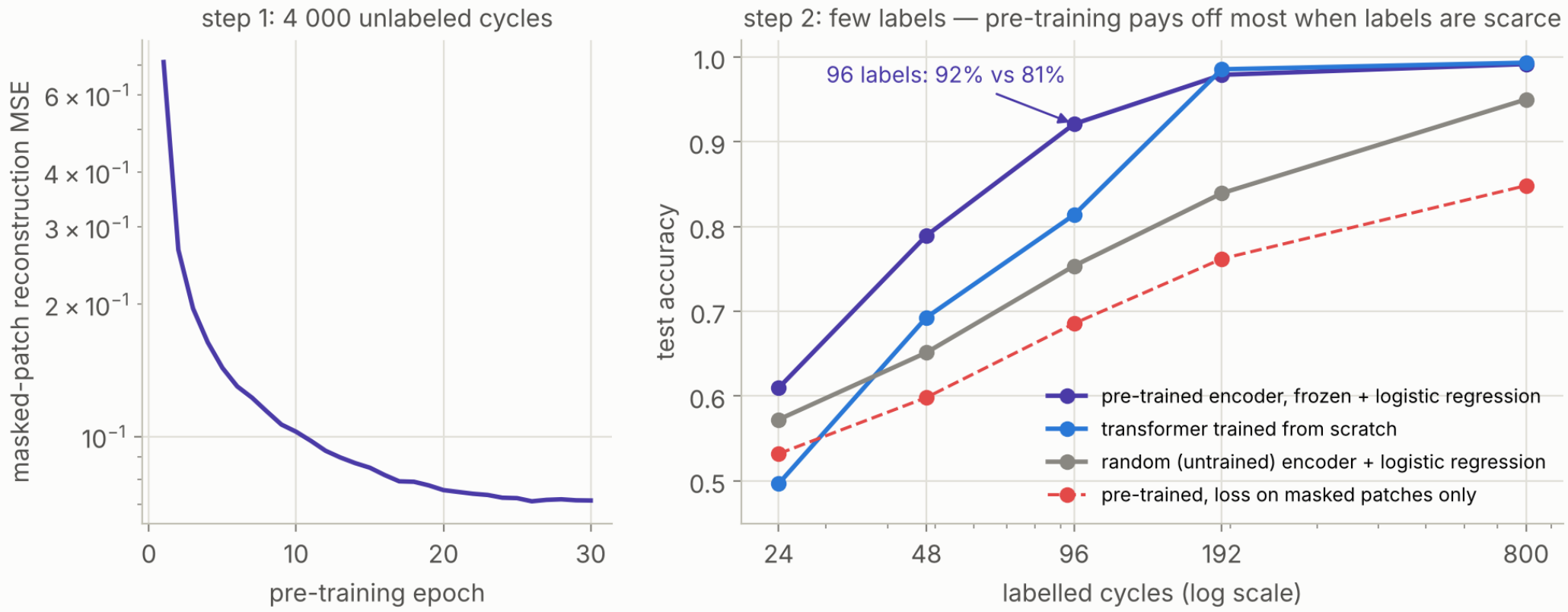
Since 2022 (check current docs)

- Decoder-only models dominate; instruction tuning and RLHF turned next-token predictors into assistants.
- Architecture converged: Pre-LN / RMSNorm, rotary positions, SwiGLU feed-forward, grouped-query attention.
- FlashAttention: exact attention computed in on-chip tiles — long contexts become affordable.
- LoRA and other parameter-efficient methods adapt a frozen model by training small low-rank updates.
- BERT's next-sentence task was dropped (RoBERTa); version pins such as transformers 4.9.1 are long obsolete.

Pick the family by the job: an encoder to classify maintenance logs, a decoder to draft the shift report.

Pre-train on unlabeled cycles, then learn from a few parts

- Every part logs a signal; few are measured on the CMM. Pre-train on 4,000 unlabeled cycles by masked-patch reconstruction (BERT's idea for signals), then freeze the encoder and fit a logistic regression.



With 96 labels: 92% pre-trained vs 81% from scratch. Loss on masked patches only: 69% — worse than a random encoder (75%).

For text, download the encoder and fine-tune it

- Maintenance logs and non-conformance reports are text: start from a pre-trained encoder (e.g. DistilBERT) instead of pre-training yourself. The modern recipe (runs on your machine; needs the Hugging Face Hub):

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification, TrainingArguments, Trainer
tok   = AutoTokenizer.from_pretrained("distilbert-base-uncased")
model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased", num_labels=3)
ds    = ds.map(lambda b: tok(b["text"], truncation=True, padding="max_length", max_length=64), batched=True)
# optional LoRA: model = get_peft_model(model, LoraConfig(task_type="SEQ_CLS", r=8, target_modules=["q_lin", "v_lin"]))
args  = TrainingArguments("log-classifier", num_train_epochs=3, learning_rate=5e-5, eval_strategy="epoch")
Trainer(model=model, args=args, train_dataset=ds["train"], eval_dataset=ds["test"]).train()
```

Feature-based (freeze)

- Run the frozen encoder once, train a small classifier on its [CLS] or pooled outputs.
- Best with tens of labels; cheap; what slide 11 did.

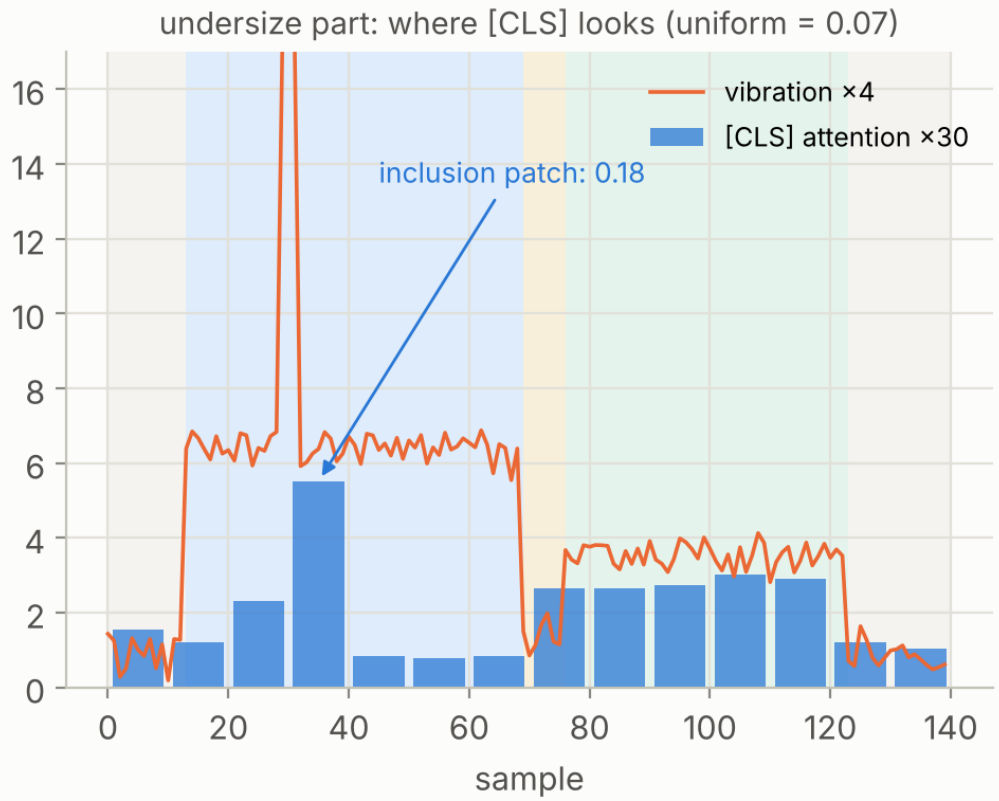
Fine-tune (update)

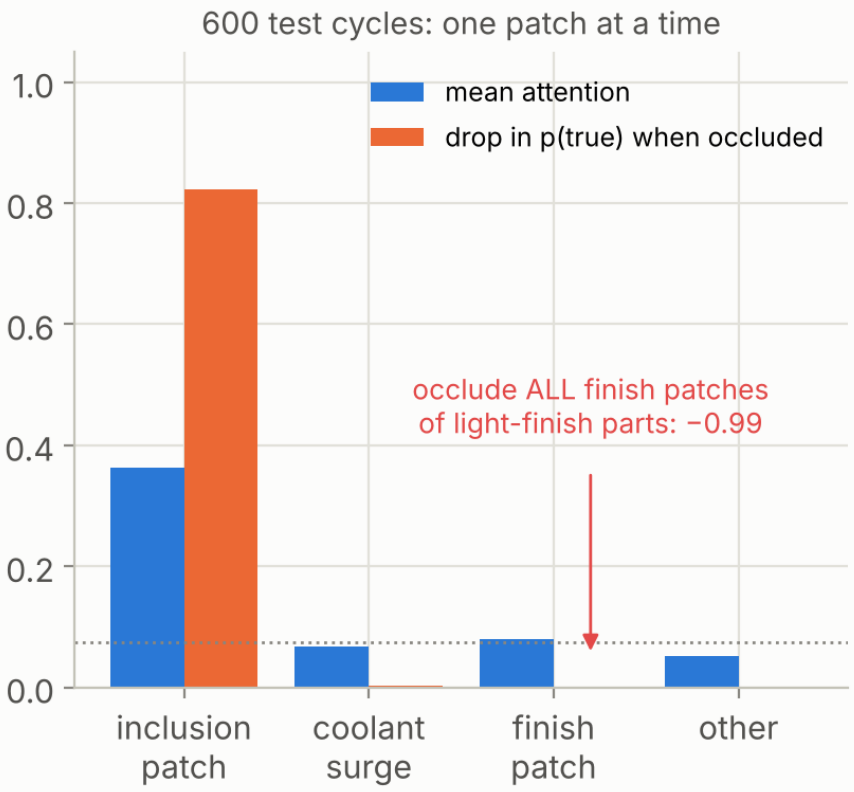
- Update all weights (or LoRA adapters) with a small learning rate, ~2–4 epochs.
- Wins once labels number in the hundreds; watch for overfitting.

Same pattern as the CNC encoder: a pre-trained body, a small task head on [CLS], a short adaptation run.

Attention shows routing, not importance — verify by occlusion

- [CLS] puts 0.36 of its last-layer attention on the inclusion patch (uniform ≈ 0.075), and occluding that patch drops p(true class) by 0.82: here the two agree.

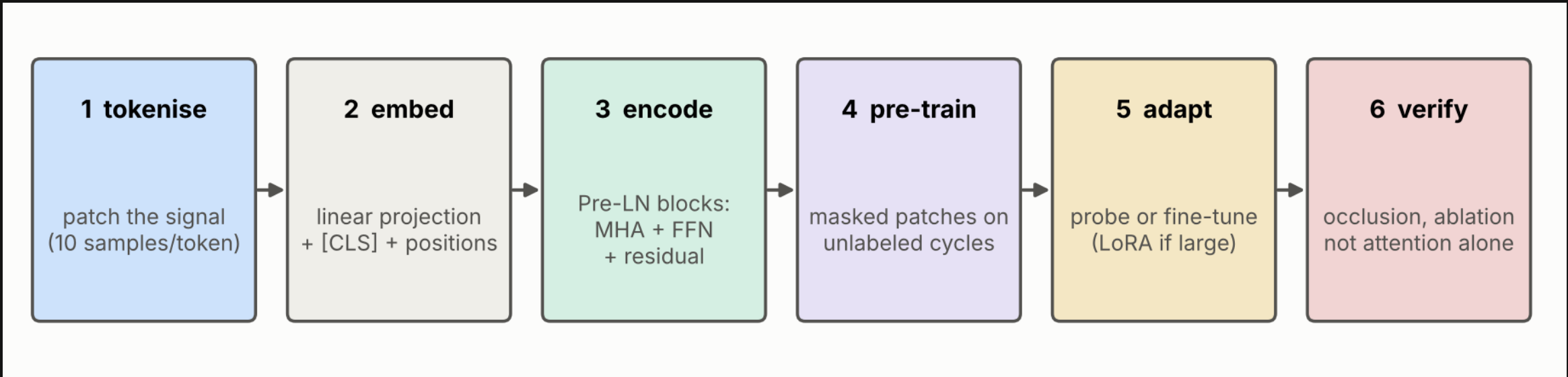




Finish patches look irrelevant to both methods, yet hiding all of them drops light-finish parts by 0.99. Redundancy hides importance; intervene before you explain.

WHAT TO TAKE AWAY

A transformer workflow for machine signals



1. Tokenise deliberately — patch the signal; attention cost grows with tokens².
2. Scale scores by $1/\sqrt{d_k}$; use Pre-LN, residuals and a key-padding mask.
3. Give positions when order matters — and test with a shuffle whether it does.
4. Mask the future in anything that forecasts; check past outputs do not move.
5. Pre-train on the cycles you already log, with an objective that keeps rare events.
6. Compare against the strong baseline on accuracy and cost.
7. Read attention as routing; confirm importance with (group) occlusion.

Notebook
`Mlpi26_lect11_nbsup1.ipynb`
17 sections · 8 exercises · references from Bahdanau (2015) and Vaswani (2017) to PatchTST and LoRA

Generated by Claude and customized by

dew.ninja 2026